MICA: Measurement, Instrumentation, Control and Analysis

Wiener System Identification: Cuber

Ariel Mobius, MIT, 5 May 2026

Revisions

2026 Jan 24 Started drawing on previous Notebooks

Keywords

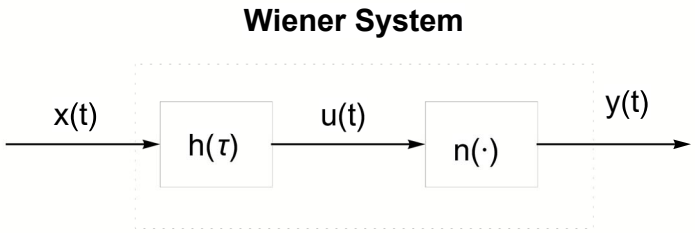
Black Box, Frequency Response, Transfer Function, Laplace Transform, Inverse Laplace Transform, Bode Plot, Gain, Phase, Filter, System ID, Sallen-Key Filter, Low-Pass System, High-Pass System, Second-Order System, Gain, Natural Frequency, Damping Parameter, Hertz (Hz), Radians/s, Impulse Response Function, Resistor, Capacitor, Inductor, Step Response, Electrical Impedance, Minimization, Objective Function, Parameter Estimation, Least-Squares, Levenberg-Marquardt Algorithm, Step Response, Network Analyzer, Swept Sine, Stochastic Input, Stochastic Response, Convolution, Prediction Error, Variance Accounted For (VAF), Parametric Model, Non-Parametric Model, Training Set, Prediction Set, Residuals

Objectives

The purpose of this MICA Module is to define Wiener systems and introduce techniques which under certain circumstances enables Wiener systems to be characterized experimentally.

Wiener systems

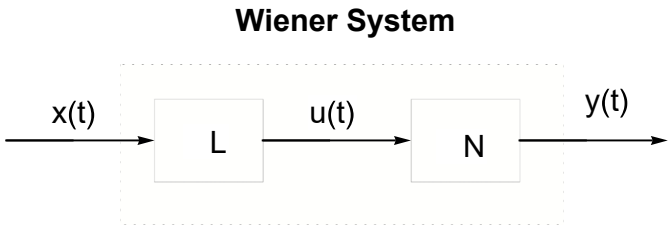
A linear dynamic subsystem followed by a static nonlinearity (e.g., cuber, hard limiter, etc) is called a Wiener system after Norbert Wiener (see https://en.wikipedia.org/wiki/Norbert_Wiener), the MIT mathematician who developed a number of system identification techniques. Wiener systems should not be confused with Wiener kernel nonlinear system representations.



Here we see a Wiener system consisting of a linear dynamic subsystem as represented by an impulse response function, $h(\tau)$, follow by a static nonlinear element, $n(\cdot)$. The input to the Wiener system is

denoted by, x(t), and the output from the Wiener system is denoted by, y(t). An internal signal u(t) is the output from the linear dynamic component, h(τ), and is also the input to the static nonlinear element, n(·).
Sometimes Wiener systems are called LN systems because there is a linear dynamic system (L) followed by a static nonlinearity (N).

An NL system consists of a nonlinear static element (N) followed by a linear dynamic element (L) and is called a Hammerstein system. Hammerstein system identification is the subject of another MICA Notebook.



In this MICA Notebook we will simulate a Wiener system and attempt to discover, h(τ) and n(·) purely from the input, x(t), and output, y(t), data sets.

Note

You can progress through this Notebook by clicking on the ■ at the start of each region of code. This will execute (evaluate) the region of code associated with ■ and any results of the code execution will be generated and added to the Notebook. After inspecting the results and reading subsequent text you can use the mouse scroll wheel (if available) to scroll down to the next ■ .

The whole Notebook can be executed by selecting Evaluation from the menu above and clicking on Evaluate Notebook. While it is OK to do this here, it can be potentially disastrous if this is done in MICA Notebooks containing code to run experiments.

In[ ]:=
```
■; Clear["Global`*"]
```

Create Wiener system

As an example lets start by generating an impulse response function, h(τ), (to model the linear dynamic element) and lets make the nonlinear static element, n(·), a cuber.

Generate h(τ)

Lets choose a second – order low –
pass subsystem. The general transfer function for such a sub system is :

In[ ]:=
```
■; H[s_, α_, ω_, ζ_] := α ω^2 / (s^2 + 2 ζ ω s + ω^2)
```

If we inverse Laplace transform this transfer function then we get:

```
In[ ]:= InverseLaplaceTransform[H[s, α, ω, ζ], s, τ] // FullSimplify
```

Out[ ]=

$$\frac{e^{-\zeta\,\tau\,\omega}\,\alpha\,\omega\,\text{Sinh}\left[\sqrt{-1+\zeta^2}\;\tau\,\omega\right]}{\sqrt{-1+\zeta^2}}$$

This is the system continuous (or symbolic) unit impulse response function which we will denote h in order to distinguish it from the sampled version (see below) which we will denote, h. We also denote the continuous lag domain by τ to distinguish it from the sampled version (see below) which is denoted by τi.

```
In[ ]:= h[τ_, α_, ω_, ζ_] := e^(-ζ τ ω) α ω Sinh[Sqrt[-1 + ζ^2] τ ω] / Sqrt[-1 + ζ^2]
```

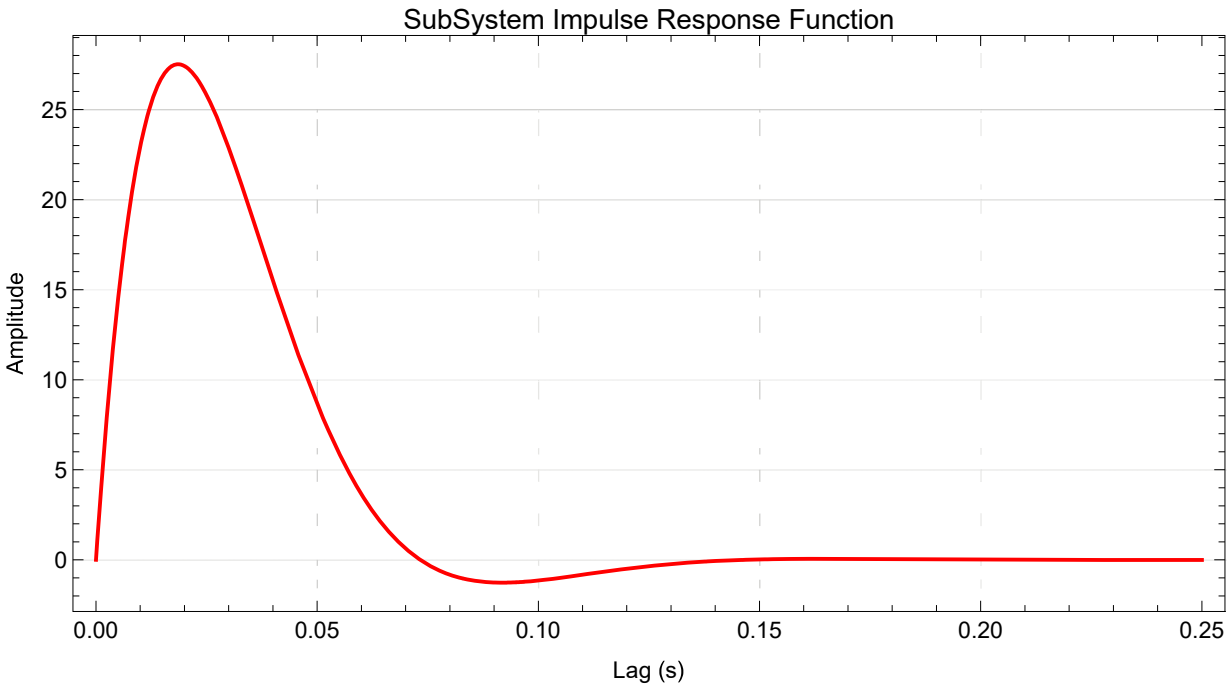
Lets make the static gain, $\alpha = 1$, the natural frequency, $\omega = 60$ rad/s (around 10 Hz), and the damping parameter $\zeta = 0.7$.

```
In[ ]:= h[τ_] := h[τ, 1, 60, 0.7] // FullSimplify // Re
```

Lets plot this:

```
In[ ]:= Plot[h[τ], {τ, 0, 0.25}, GridLines → Automatic, PlotStyle → Red,
  Frame → True, LabelStyle → {12, Black}, FrameLabel → {"Lag (s)", "Amplitude"},
  AspectRatio → 0.5, PlotLabel → "SubSystem Impulse Response Function",
  PlotRange → All, ImageSize → Automatic → 600]
```

Out[ ]=



Notice that this impulse response function “dies out” by around 0.15 s.

We will need to sample this symbolic version of the impulse response function to produce a sampled version. To do this we need to specify the inter-sample interval, $\Delta\tau$, and the maximum lag, τMax. Lets

make the sampling increment be 2 ms (i.e., a sampling rate of 500 Hz) and the maximum lag be 0.25 s (as above).

In[ ]:=
```
; Δτ = 0.002; τMax = 0.25;
```

We can now generate a sampled version of the lag domain, τ:

In[ ]:=
```
; τi = Range[0, τMax, Δτ];
```

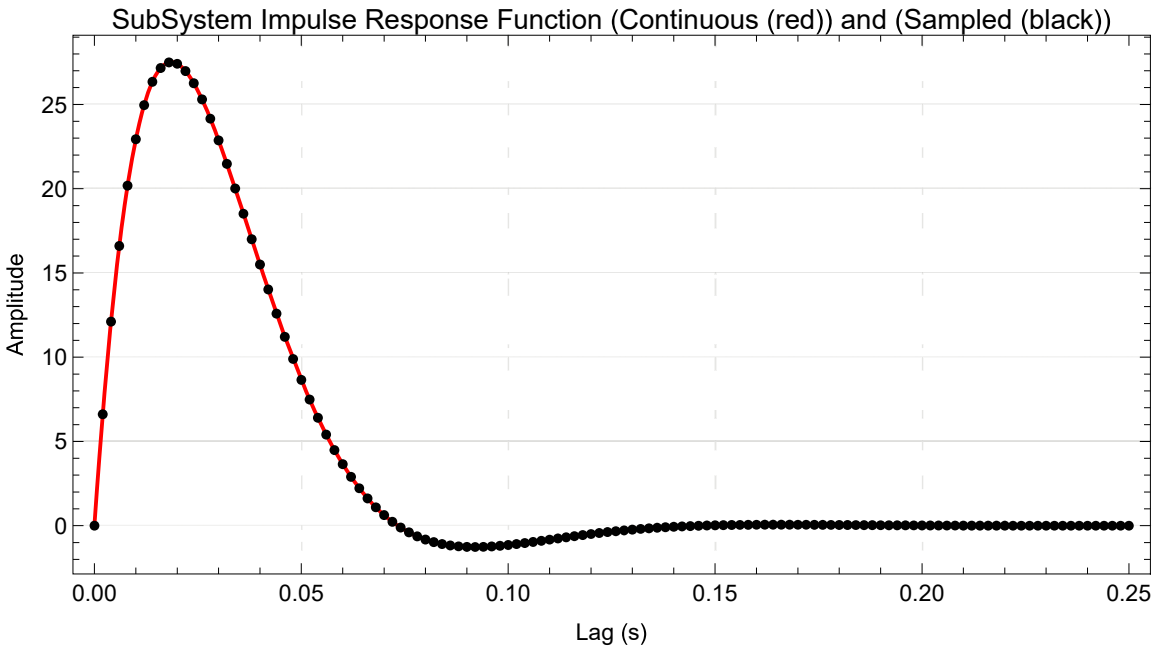
and a sampled version of the impulse response function:

In[ ]:=
```
; hi = h[τi];
```

Now we can plot both the continuous and sampled versions of h:

In[ ]:=
```
; Show[
 Plot[h[τ], {τ, 0, 0.25}, PlotStyle → Red,
  Frame → True, LabelStyle → {12, Black}, GridLines → Automatic,
  FrameLabel → {"Lag (s)", "Amplitude"}, AspectRatio → 0.5, PlotLabel →
   "SubSystem Impulse Response Function (Continuous (red)) and (Sampled (black))",
  PlotRange → All, ImageSize → 600],
 ListPlot[{τi, hi}^T, PlotStyle → Black]]
```

Out[ ]=



It is always a good idea to plot the individual points in the impulse response function to check that the sampling rate is high enough given the time course of the function. It is best to have plenty of points (e.g., > 5) covering the initial rise of the impulse response function. It is also best to have a domain (whose last value is τMax) which is 1.5 x to 2 x the time it takes for the impulse response to "die out". If these conditions are not met then resample the original continuous (symbolic) impulse response function with a more appropriate sampling interval, $\Delta\tau$, and maximum lag, τMax.

Note that the area under the impulse response function should equal the static gain, which is 1.0 in this case. Lets check by crude rectangular numeric integration:

In[ ]:=
```
; Total[hi] Δτ
```

Out[ ]=
```
0.998832
```

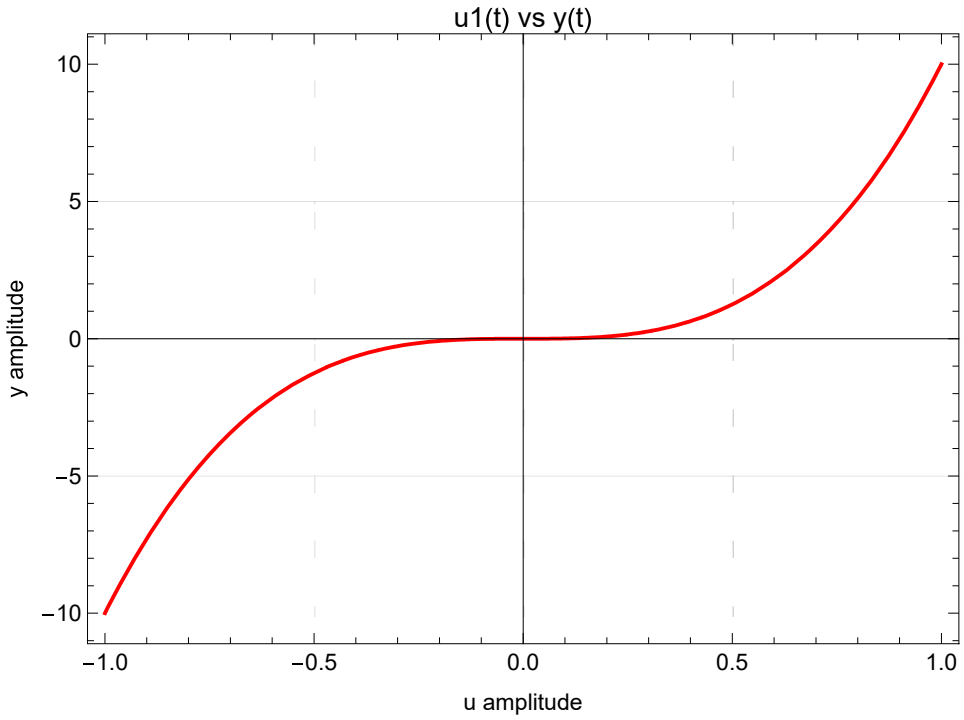
This is close enough.

Cuber static nonlinearity

In this example we will make the static nonlinearity a cuber with a scaling of 10:

In[ ]:=
```
; n[u_] := 10 u^3
Plot[n[u], {u, -1, 1}, GridLines → Automatic, PlotStyle → Red, Frame → True,
 LabelStyle → {12, Black}, FrameLabel → {"u amplitude", "y amplitude"},
 AspectRatio → 0.7, PlotLabel → "u1(t) vs y(t)", PlotRange → All, ImageSize → 500]
```

Out[ ]=



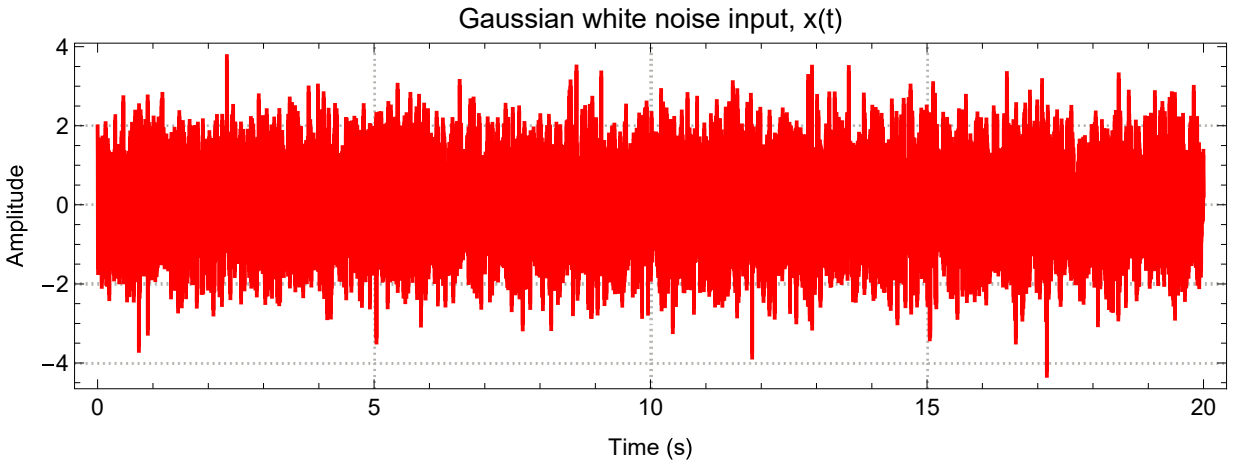
Generate the input, x(t), the unobservable internal signal, u(t) and the output, y(t)

Generate Gaussian white noise signal, x(t)

We start by generating 20 seconds of a Gaussian white noise input, x(t), sampled at around 500 Hz, with a mean of 0 and a standard deviation of 1. Note that we could have used pretty much any stochastic input here except a stochastic binary input (which is not sufficient to identify a nonlinear system). Note that as before we generate a sampled version of the time domain and specify a maximum time, tMax. Note also that we use the same sampling interval in time, Δt, as we used for lag, Δτ.

In[ ]:=
```
■; tMax = 20.0;
Δt = Δτ;
ti = Range[0.0, tMax, Δt];
x = Table[RandomVariate[NormalDistribution[0, 1]], Length[ti]];
ListLinePlot[{ti, x}^T, PlotTheme → "Detailed",
 PlotRange → All, PlotStyle → Red, Frame → True, LabelStyle → {12, Black},
 FrameLabel → {"Time (s)", "Amplitude"}, AspectRatio → 0.3,
 PlotLabel → "Gaussian white noise input, x(t)", ImageSize → Automatic → 600]
```

Out[ ]=

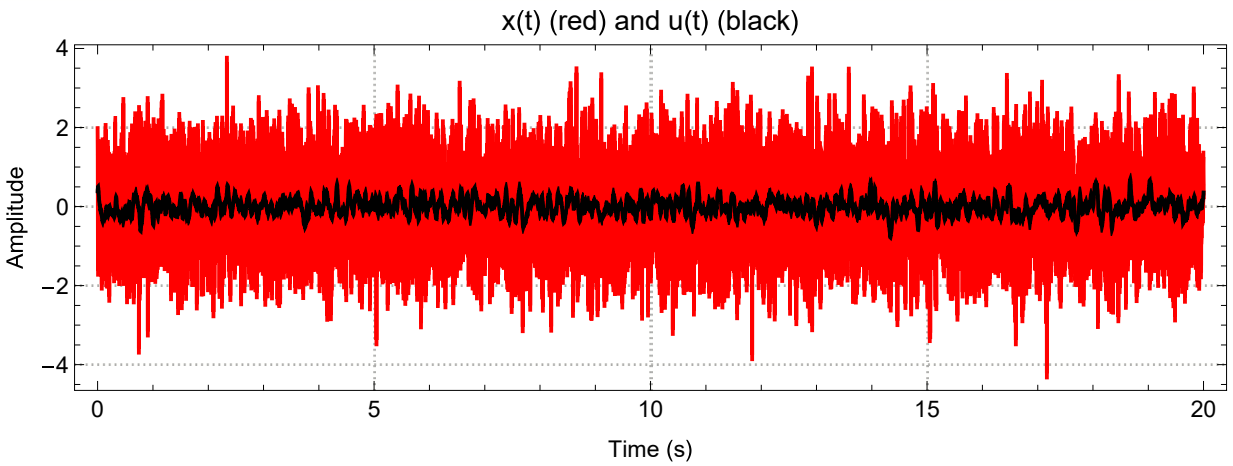


Convolve, h(τ), with x(t) to produce, u(t)

We now numerically convolve our sampled input, x(t), with the sampled version of our impulse response function, h(τ), to generate the internal signal, u(t):

In[ ]:=
```
■; u = Δt ListConvolve[hi, x, {1, 1}] ;
ListLinePlot[{{ti, x}^T, {ti, u}^T}, PlotTheme → "Detailed", PlotRange → All,
 PlotStyle → {Red, Black}, Frame → True, LabelStyle → {12, Black},
 FrameLabel → {"Time (s)", "Amplitude"}, AspectRatio → 0.3,
 PlotLabel → "x(t) (red) and u(t) (black)", ImageSize → Automatic → 600]
```

Out[ ]=

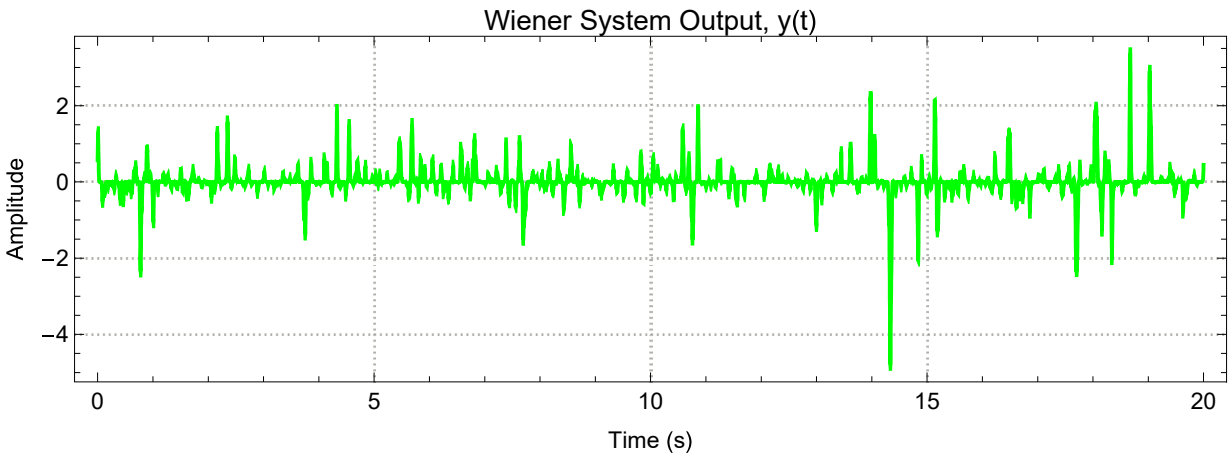


Generate output, y(t)

We now pass u(t) through our static nonlinearity, n(), to produce the output, y(t):

In[ ]:=
```
; y = n[u];
ListLinePlot[{ti, y}^T, PlotTheme → "Detailed", PlotRange → All, PlotStyle → Blue,
 Frame → True, LabelStyle → {12, Black}, FrameLabel → {"Time (s)", "Amplitude"},
 AspectRatio → 0.3, PlotLabel → "Wiener System Output, y(t)", ImageSize → Automatic → 600]
```

Out[ ]=



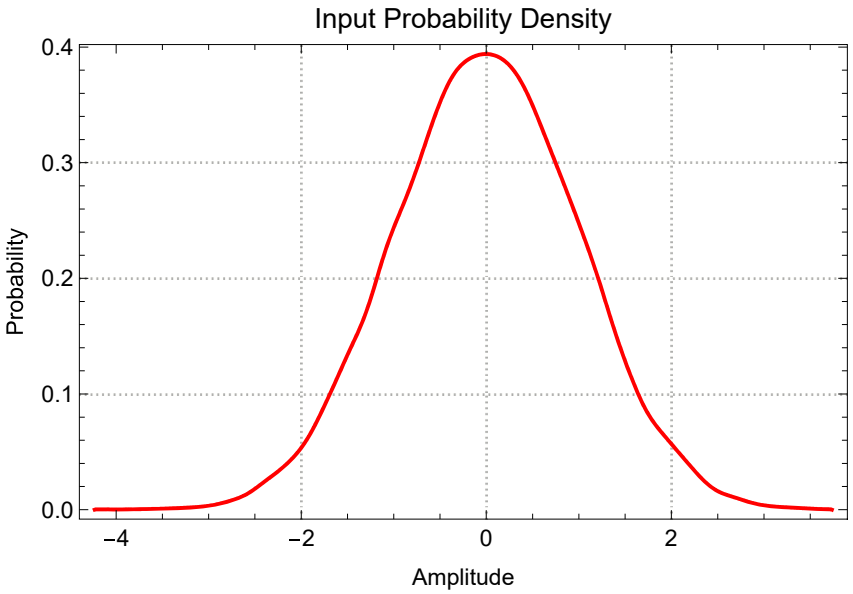
Compare input and output probability density functions

We now calculate the input and output probability density functions using Mathematica's SmoothHistogram[ ] function which determines the probability density functions via a different technique than the usual Histogram[ ] function. There is a whole area of statistics devoted to different probability density function determination methods.

Notice that while the original input, x(t) is Gaussian:

In[ ]:=
```
; SmoothHistogram[x, Automatic, "PDF", PlotRange → All, PlotTheme → "Detailed",
 FrameLabel → {"Amplitude", "Probability"}, LabelStyle → {12, Black}, PlotStyle → Red,
 PlotLabel → "Input Probability Density", ImageSize → Automatic → 400]
```

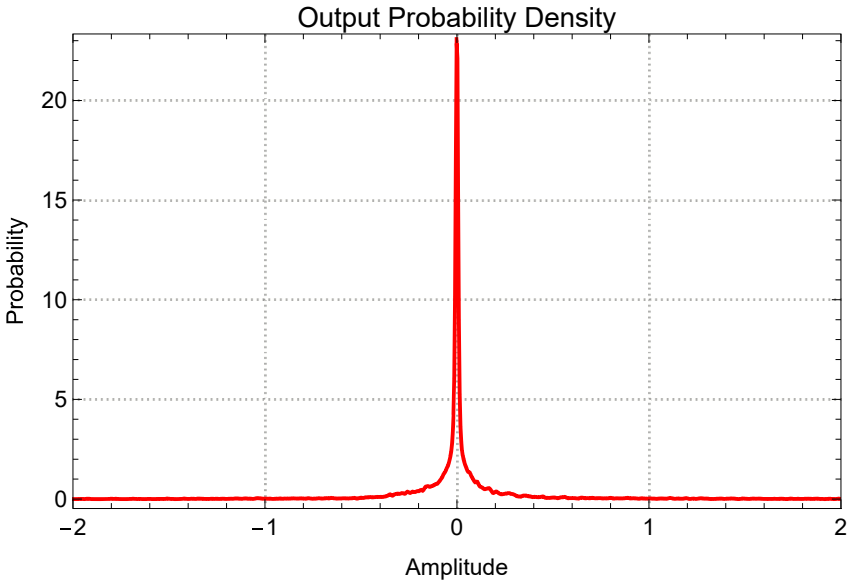
Out[ ]=



The output, y(t), is clearly nonGaussian:

In[ ]:=
```
; SmoothHistogram[y, Automatic, "PDF", PlotRange → {{-2, 2}, All}, PlotTheme → "Detailed",
 FrameLabel → {"Amplitude", "Probability"}, LabelStyle → {12, Black}, PlotStyle → Red,
 PlotLabel → "Output Probability Density", ImageSize → Automatic → 400]
```

Out[ ]=



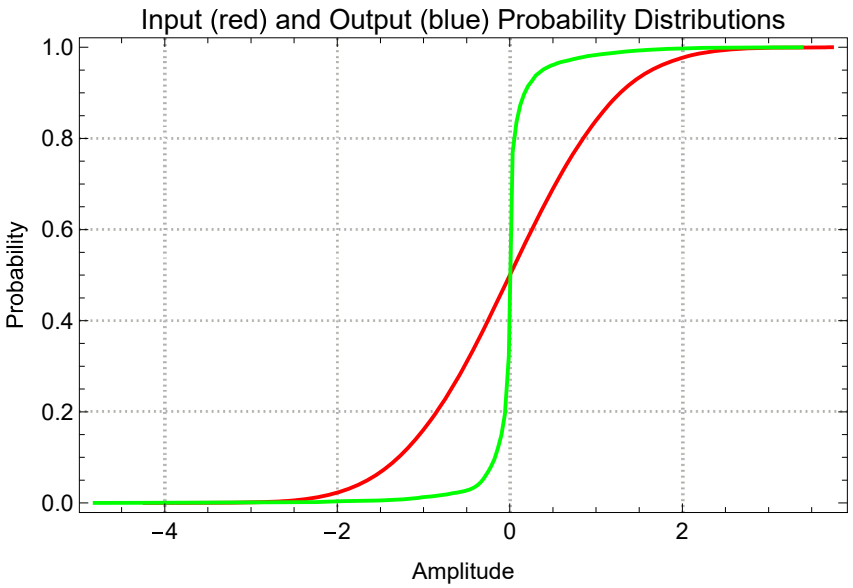
A linear system perturbed by a Gaussian stochastic input (does not need to be white) will produce a Gaussian stochastic output. If the output from a system driven by a Gaussian stochastic input is non Gaussian (as here) then this is clear evidence that the system is nonlinear.

Compare input and output probability distribution functions

Lets calculate the smoothed input and output probability distribution functions:

In[ ]:=
```
; Show[
 SmoothHistogram[x, Automatic, "CDF", PlotStyle → Red, PlotRange → All, PlotTheme → "Detailed",
  FrameLabel → {"Amplitude", "Probability"}, LabelStyle → {12, Black},
  PlotLabel → "Input (red) and Output (blue) Probability Distributions",
  ImageSize → Automatic → 400],
 SmoothHistogram[y, Automatic, "CDF", PlotStyle → Blue]]
```

Out[ ]=



Wiener system identification

We now ask if it is possible to determine the two internal subsystems, namely, h(τ) and n(·) solely from the input, x(t), and output, y(t), signals.
Lets see what happens when we compute the nonparametric impulse response function, hi1(τ), between the input, x(t), and the output, y(t).

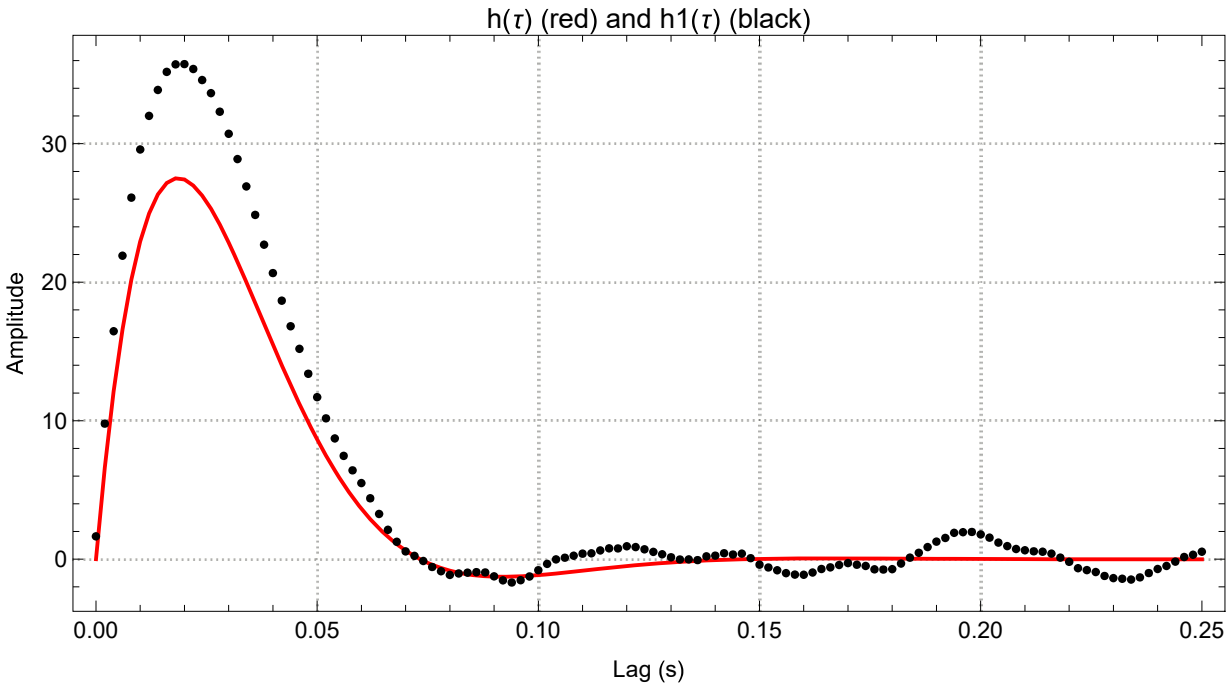
To do this we compute the input auto-correlation function, xacf(τ), and the cross-correlation function between x(t) and y(t), xyccf(τ).

Define a function to perform biased auto- and cross-correlation functions (strictly speaking it calculates auto- and cross-covariance functions)

In[ ]:=
```
; cor[x_, y_, m_] := With[{n = Length[x], xMn = Mean[x], yMn = Mean[y]},
  Table[((x[[1 ;; n - j]] - xMn).(y[[1 + j ;; n]] - yMn)) / n, {j, 0, m}] ]
```

In[ ]:=
```
; xacf = cor[x, x, τMax / Δt];
xyccf = cor[x, y, τMax / Δt];
toe = ToeplitzMatrix[xacf];
invtoe = Inverse[toe];
hi1 = (invtoe.xyccf) / Δt;
ListPlot[{{τi, hi}ᵀ, {τi, hi1}ᵀ}, Joined → {True, False},
 PlotTheme → "Detailed", PlotStyle → {Red, Black}, Frame → True,
 LabelStyle → {12, Black}, FrameLabel → {"Lag (s)", "Amplitude"}, AspectRatio → 0.5,
 PlotLabel → "h(τ) (red) and h1(τ) (black)", PlotRange → All, ImageSize → Automatic → 600]
```

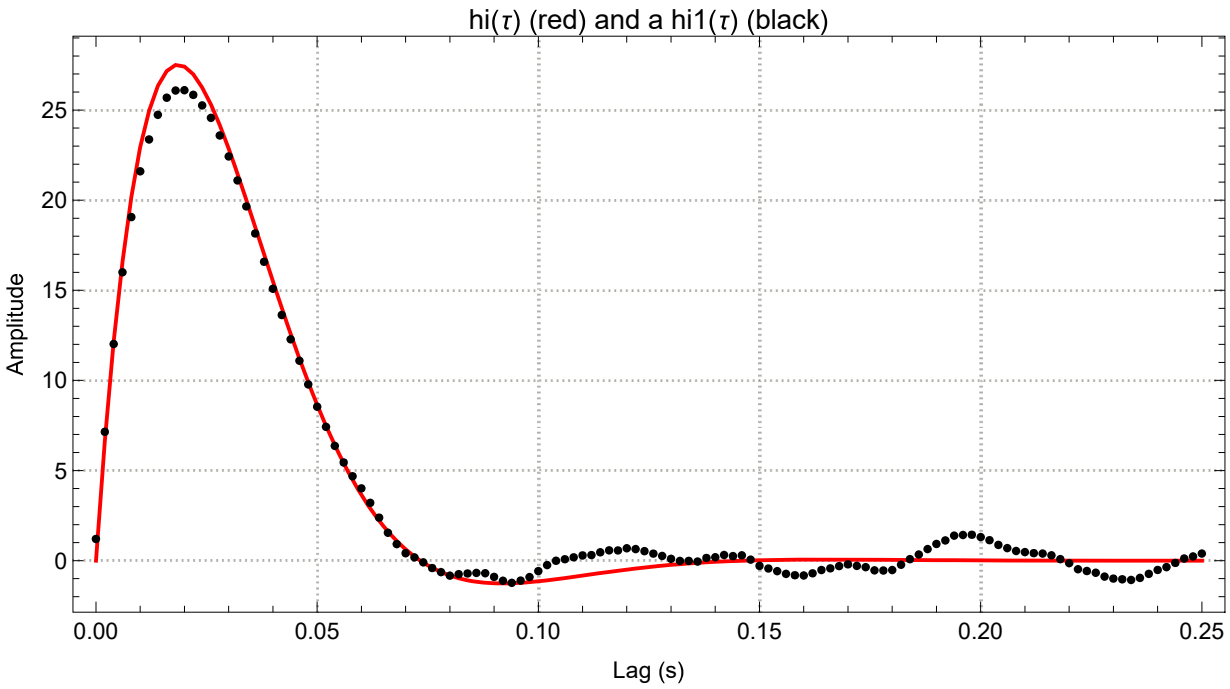
Out[ ]=



If we multiply h(τ) by α and divide n(·) by α there will be no change in the output, y(t). In other words there is a scaling constant, α , that can freely move from h(τ) to n(·). So lets scale h1(τ) so that the area under it (i.e. the static gain) is 1:

In[ ]:=
```
■; hi1 = hi1 / (Δτ Total[hi1]); (* scales hi1 to have unity static gain *)
ListPlot[{{τi, hi}ᵀ, {τi, hi1}ᵀ}, Joined → {True, False},
 PlotTheme → "Detailed", PlotStyle → {Red, Black}, Frame → True,
 LabelStyle → {12, Black}, FrameLabel → {"Lag (s)", "Amplitude"},
 AspectRatio → 0.5, PlotLabel → "hi(τ) (red) and a hi1(τ) (black)",
 PlotRange → All, ImageSize → Automatic → 600]
```

Out[ ]=



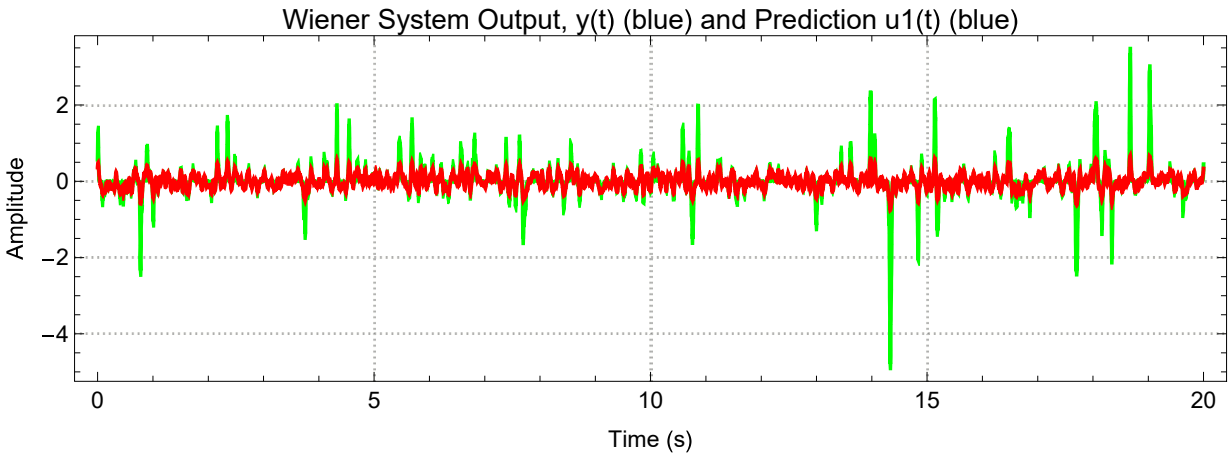
This is a remarkable result!! It shows that despite the presence of a major static nonlinearity such as a cuber, the stochastic system identification techniques were able to nevertheless get an estimate of the Wiener system's dynamic linear element. This amazing result (for the case of cross-correlation) seems to have been first discovered by Julian Bussgang (https://en.wikipedia.org/wiki/Julian_J._Bussgang) in 1951 during his Master's thesis research when he was a student at the Research Lab for Electronics (RLE) at MIT (https://en.wikipedia.org/wiki/Bussgang_theorem).

Predict u(t)

Now that we have h1(τ), we can convolve it with the input, x(t), to produce a prediction of the output but as we will shortly be assuming that the system is possibly a Wiener system this prediction can also be thought of as a prediction, u1(t), of the internal signal, u(t):

In[ ]:=
```
■; u1 = Δt ListConvolve[hi1, x, {1, 1}] ;
ListLinePlot[{{ti, y}^T, {ti, u1}^T}, PlotTheme → "Detailed",
 PlotRange → All, PlotStyle → {Blue, Red}, Frame → True, LabelStyle → {12, Black},
 FrameLabel → {"Time (s)", "Amplitude"}, AspectRatio → 0.3,
 PlotLabel → "Wiener System Output, y(t) (blue) and Prediction u1(t) (blue)",
 ImageSize → Automatic → 600]
```

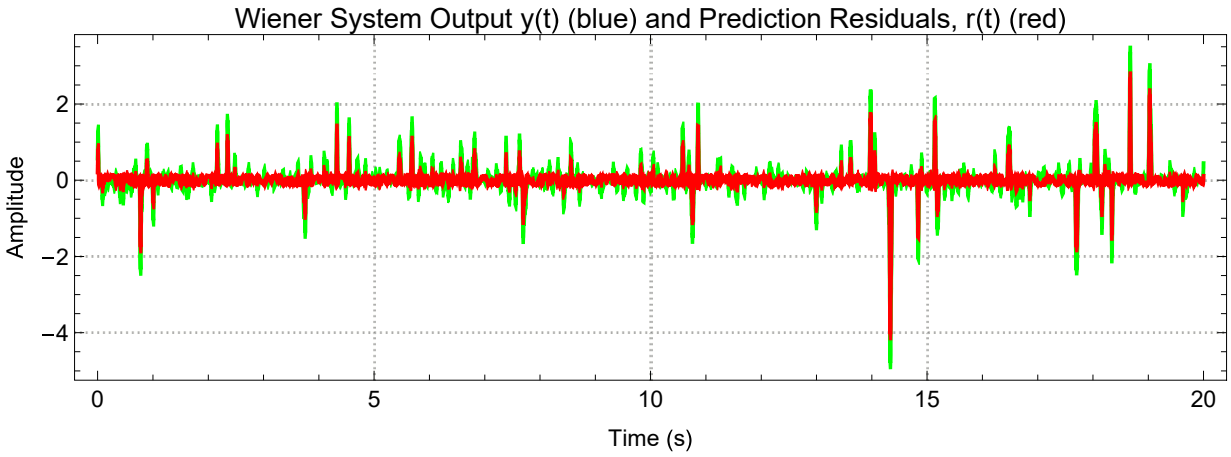
Out[ ]=



The prediction is not at all that good. Lets calculate the residuals, r1(t):

In[ ]:=
```
■; r1 = y - u1;
ListLinePlot[{{ti, y}^T, {ti, r1}^T}, PlotTheme → "Detailed",
 PlotRange → All, PlotStyle → {Blue, Red}, Frame → True, LabelStyle → {12, Black},
 FrameLabel → {"Time (s)", "Amplitude"}, AspectRatio → 0.3,
 PlotLabel → "Wiener System Output y(t) (blue) and Prediction Residuals, r(t) (red)",
 ImageSize → Automatic → 600]
```

Out[ ]=



In[ ]:=
```
■; vaf0 = 100.0 (1 - Variance[r1] / Variance[y])
```

Out[ ]=

```
54.3523
```

This is a low variance accounted for (VAF) and unless we think that around 40 to 50% of the output is noise, we should continue our analysis and consider a nonlinear model. If our system was dynamically

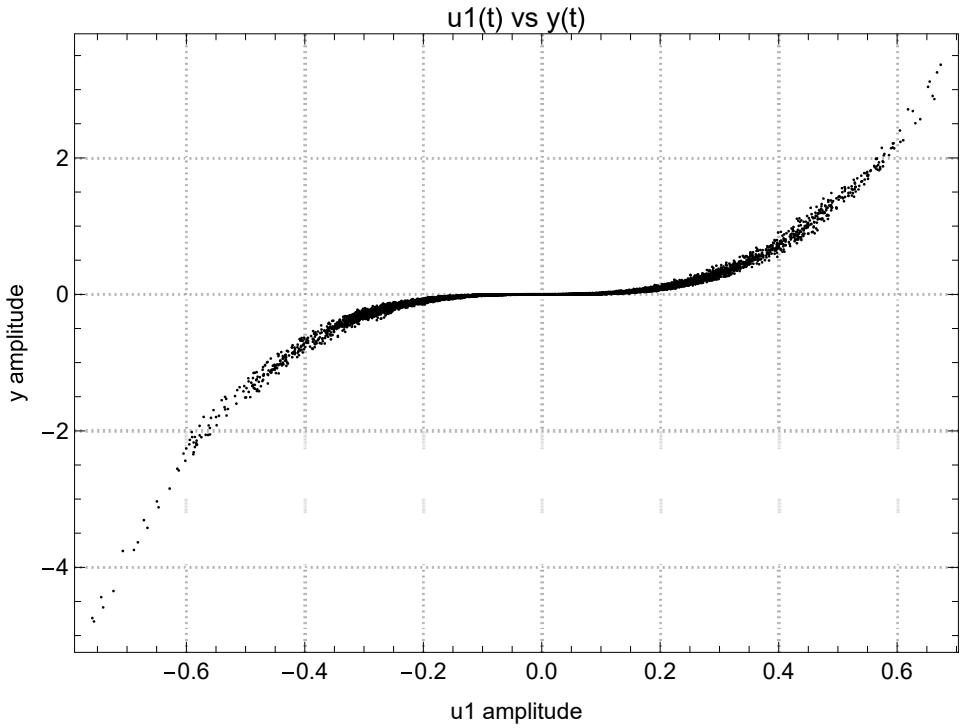
linear then if we were to plot our prediction of y (u1 in this case) against the actual output y then we should get a straight line with scatter about this in proportion to the noise.

Predict n(·)

So lets cross-plot u1 vs y:

In[ ]:=
```
; ListPlot[{u1, y}ᵀ, PlotTheme → "Detailed", PlotStyle → {Red, Black}, Frame → True,
 LabelStyle → {12, Black}, FrameLabel → {"u1 amplitude", "y amplitude"},
 AspectRatio → 0.7, PlotLabel → "u1(t) vs y(t)", PlotRange → All, ImageSize → 500]
```

Out[ ]=



This is clearly not a straight line and immediately suggests that at the very least we have Wiener system (i.e., LN system).

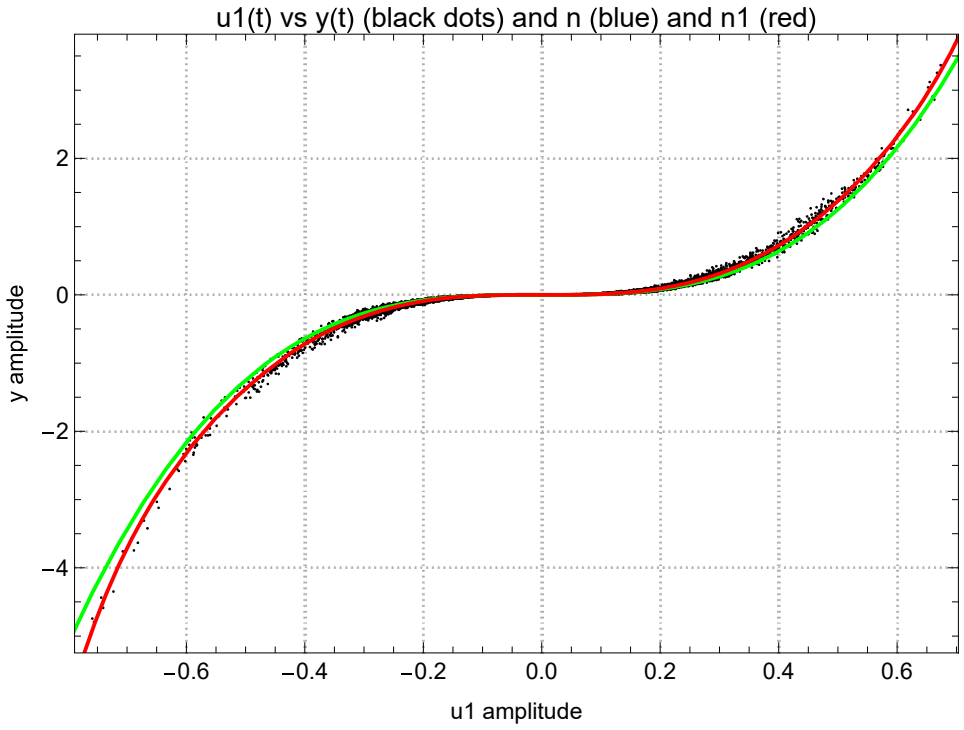
We can find an approximation to the estimate, n1(·), of n(·) above by fitting a high-order polynomial to the u1(t) vs y(t) cross-plot. We are using the polynomial here as a type of nonparametric function approximation but because we need to subsequently invert this function we will fit a polynomial that only has odd powers (the fitted function needs to be either strictly monotonically increasing or decreasing in order to have a unique inverse). Note that using odd powers does not ensure a monotonic fit but increases our odds (Note: we really need to replace the use of these polynomials with another approach which ensures monotonicity):

In[ ]:=
```
; n1 = LinearModelFit[{u1, y}ᵀ, {1, uu, uu³, uu⁵, uu⁷, uu⁹}, uu]["Function"];
```

Lets plot this estimate, n1(·), of the nonlinearity together with the cross-plot of u1(t) vs y(t):

In[ ]:=
```
; Show[
 ListPlot[{u1, y}^T, PlotTheme → "Detailed", PlotStyle → Black, Frame → True,
  LabelStyle → {12, Black}, FrameLabel → {"u1 amplitude", "y amplitude"},
  AspectRatio → 0.7, PlotLabel → "u1(t) vs y(t) (black dots) and n (blue) and n1 (red)",
  PlotRange → All, ImageSize → 500],
 Plot[{n[uu], n1[uu]}, {uu, -1, 1}, PlotStyle → {Blue, Red}]]
```

Out[ ]=



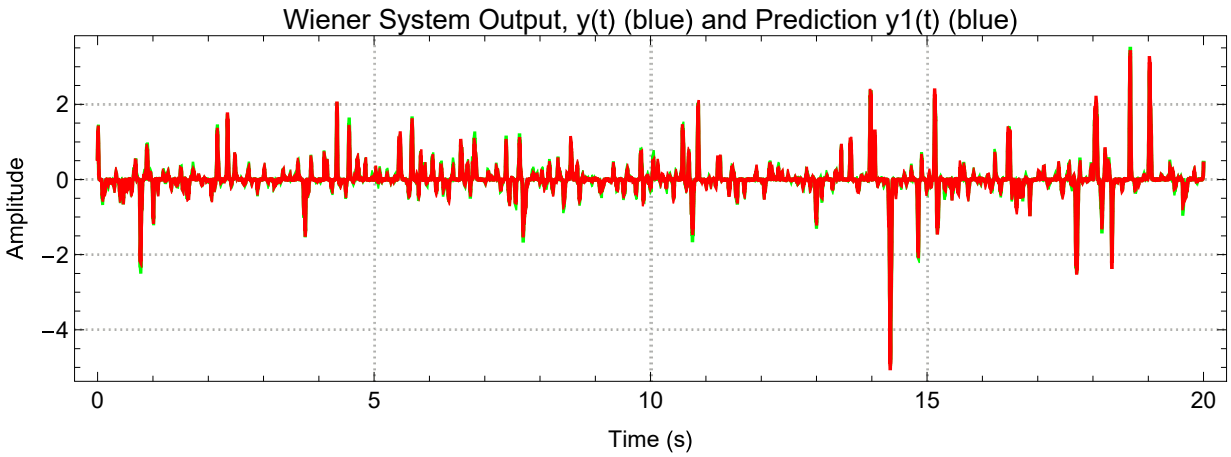
So as we can see our estimate of the Wiener system static nonlinearity, n1(·), is close to the actual static nonlinear element, n(·).

Predict output

Now that we have the estimate, n1(·), we can predict the output, y1(t), from our u1(t) estimate:

In[ ]:=
```
; y1 = n1[u1];
ListLinePlot[{{ti, y}^T, {ti, y1}^T}, PlotTheme → "Detailed",
 PlotRange → All, PlotStyle → {Blue, Red}, Frame → True, LabelStyle → {12, Black},
 FrameLabel → {"Time (s)", "Amplitude"}, AspectRatio → 0.3,
 PlotLabel → "Wiener System Output, y(t) (blue) and Prediction y1(t) (blue)",
 ImageSize → Automatic → 600]
```

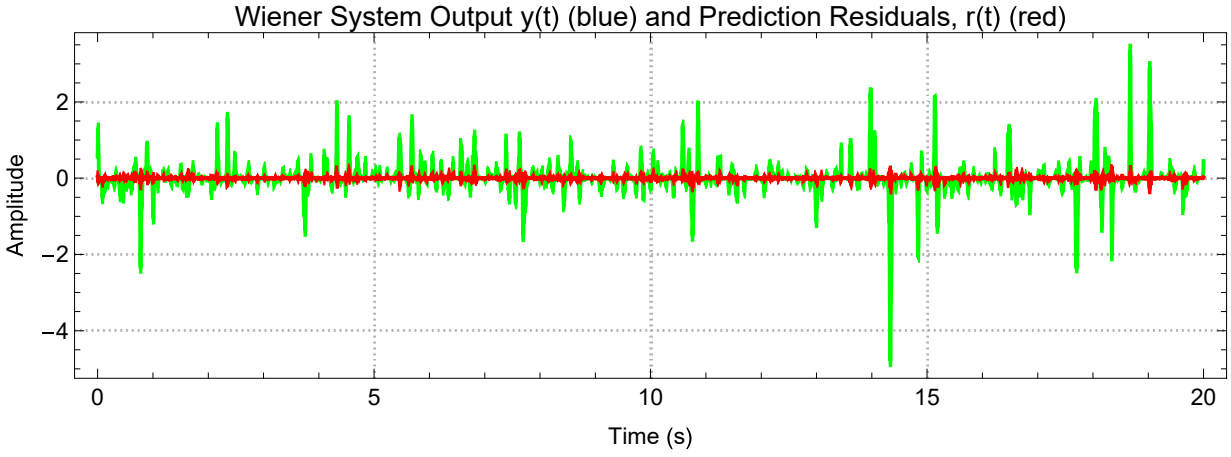
Out[ ]=



The prediction is so good it is difficult to see the difference between y(t) and y1(t). So lets calculate the residuals, r1(t):

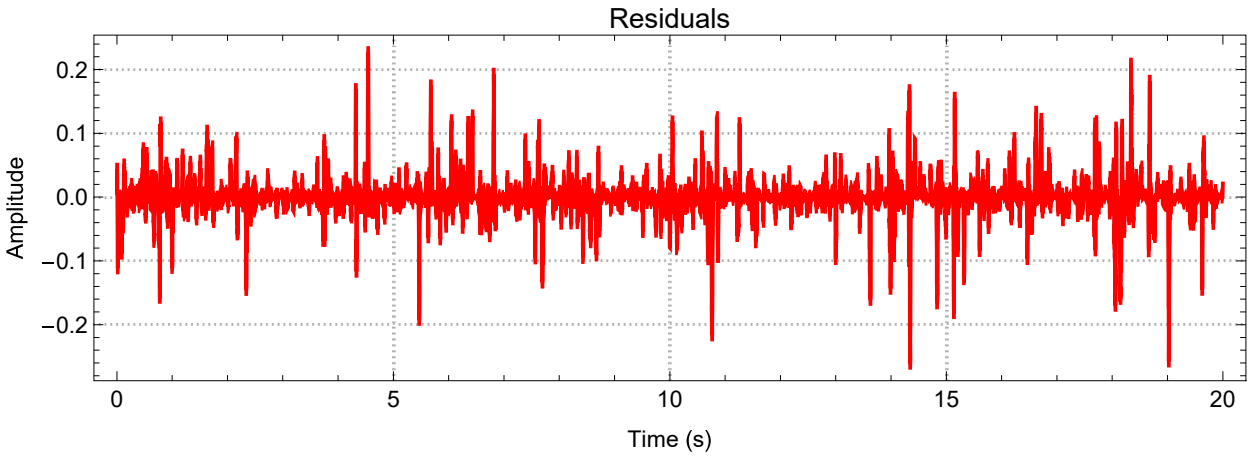
In[ ]:=
```
; r1 = y - y1;
ListLinePlot[{{ti, y}^T, {ti, r1}^T}, PlotTheme → "Detailed",
 PlotRange → All, PlotStyle → {Blue, Red}, Frame → True, LabelStyle → {12, Black},
 FrameLabel → {"Time (s)", "Amplitude"}, AspectRatio → 0.3,
 PlotLabel → "Wiener System Output y(t) (blue) and Prediction Residuals, r(t) (red)",
 ImageSize → Automatic → 600]
```

Out[ ]=



In[ ]:=
```
; ListLinePlot[{ti, r1}^T, PlotTheme → "Detailed", PlotRange → All, PlotStyle → Red,
 Frame → True, LabelStyle → {12, Black}, FrameLabel → {"Time (s)", "Amplitude"},
 AspectRatio → 0.3, PlotLabel → "Residuals", ImageSize → Automatic → 600]
```

Out[ ]=



In[ ]:=
```
; vaf1 = 100.0 (1 - Variance[r1] / Variance[y])
```

Out[ ]=

```
99.384
```

Note that the VAF is now really high!! At this point we could stop because the VAF is so high. However in this simulation of a Wiener system we did not add noise to our output (which would have decreased the VAF). We will therefor attempt to refine our estimates of h(τ) and n(·) to see if we can achieve an even higher VAF.

Function reversion (sometimes called function inversion)

There are many occasions in which given a function y = f(x) we want to find a function x = g(y). Function g() is often (including Mathematica) called the inverse function of f() and sometimes in mathematics texts it is called the reversion of f(). For example, you might have a function voltage = f(temperature) relating the voltage generated by a thermocouple junction to its temperature. However if you measure the voltage across a thermocouple junction and wish to deduce the its temperature you need the inverse function, g(), so that given a voltage you can calculate the temperature via temperature = g(voltage).

Many of the worlds most famous mathematicians have worked on the reversion problem including Gauss and Lagrange.
Mathematica has some powerful function reversion capabilities (under the functions InverseFunction[ ] and InverseSeries[ ].

For example, the reversion (or inverse) of y = sin (x) is x = arcsin (y)

In[ ]:=
```
; InverseFunction[Sin]
```

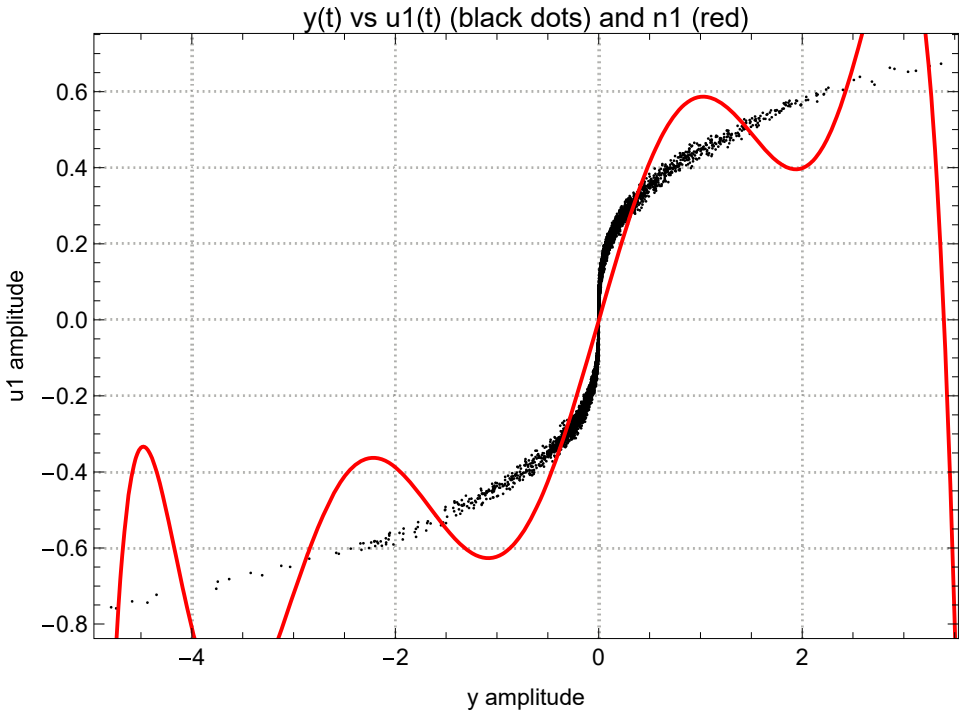
Out[ ]=
```
ArcSin
```

We would like to find the reversion of n1(·) so that given y(t) we can generate a new estimate, u2(t), of u(t).

A simple way to “invert” n1(·) is to refit the u1(t) vs y(t) but this time by reversing their roles to y(t) vs u1(t)as follows. We could use traditional polynomials like we did before:

In[ ]:=
```
■;
n1Inv =
 LinearModelFit[{y, u1}ᵀ, {1, uu, uu², uu³, uu⁴, uu⁵, uu⁶, uu⁷, uu⁸, uu⁹}, uu]["Function"];
Show[
 ListPlot[{y, u1}ᵀ, PlotTheme → "Detailed", PlotStyle → Black, Frame → True,
  LabelStyle → {12, Black}, FrameLabel → {"y amplitude", "u1 amplitude"},
  AspectRatio → 0.7, PlotLabel → "y(t) vs u1(t) (black dots) and n1 (red)",
  PlotRange → All, ImageSize → 500],
 Plot[n1Inv[uu], {uu, -10, 10}, PlotStyle → Red]]
```

Out[ ]=



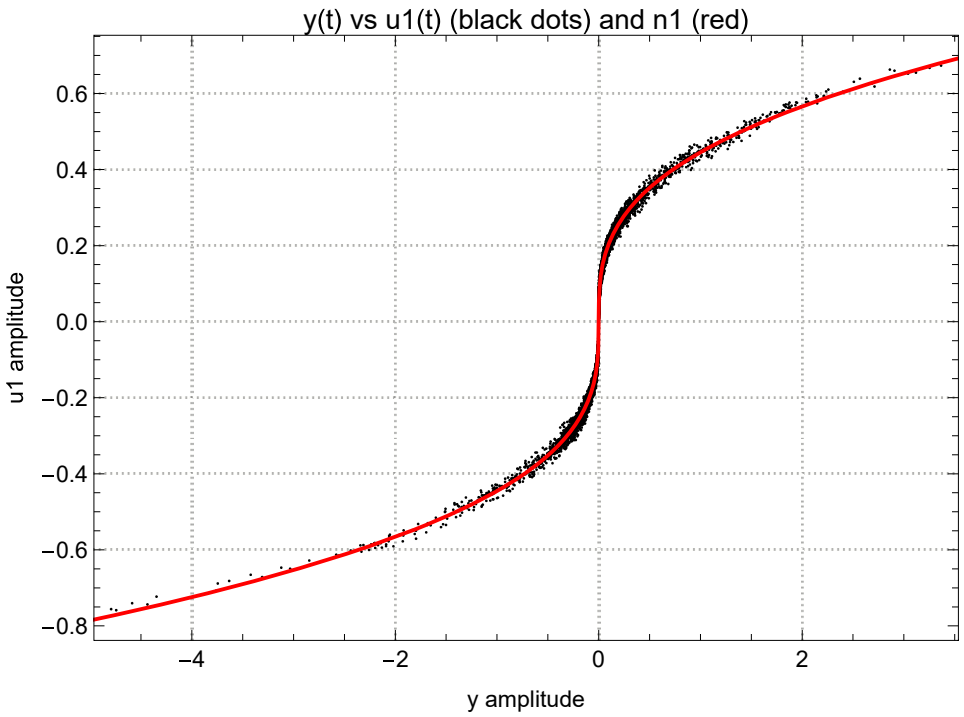
But as you see the fit is really bad.

So because the nonlinearity looks somewhat cubic lets instead use nth roots as terms in the “nonparametric” fit. As even powered roots have imaginary parts which we cannot use here we will restrict the nth root terms to have odd root powers.

Incidentally nth roots are sometimes called surd functions.

In[ ]:=
```
;
n1Inv = LinearModelFit[{y, u1}^T, {1, uu, (uu)^(1/3), (uu)^(1/5), (uu)^(1/7), (uu)^(1/9)}, uu]["Function"];
Show[
 ListPlot[{y, u1}^T, PlotTheme → "Detailed", PlotStyle → Black, Frame → True,
  LabelStyle → {12, Black}, FrameLabel → {"y amplitude", "u1 amplitude"},
  AspectRatio → 0.7, PlotLabel → "y(t) vs u1(t) (black dots) and n1 (red)",
  PlotRange → All, ImageSize → 500],
 Plot[n1Inv[uu], {uu, -10, 10}, PlotStyle → Red]]
```

Out[ ]=



This is looking much better.

Now that we have n1Inv we can generate a u2(t) from y(t):

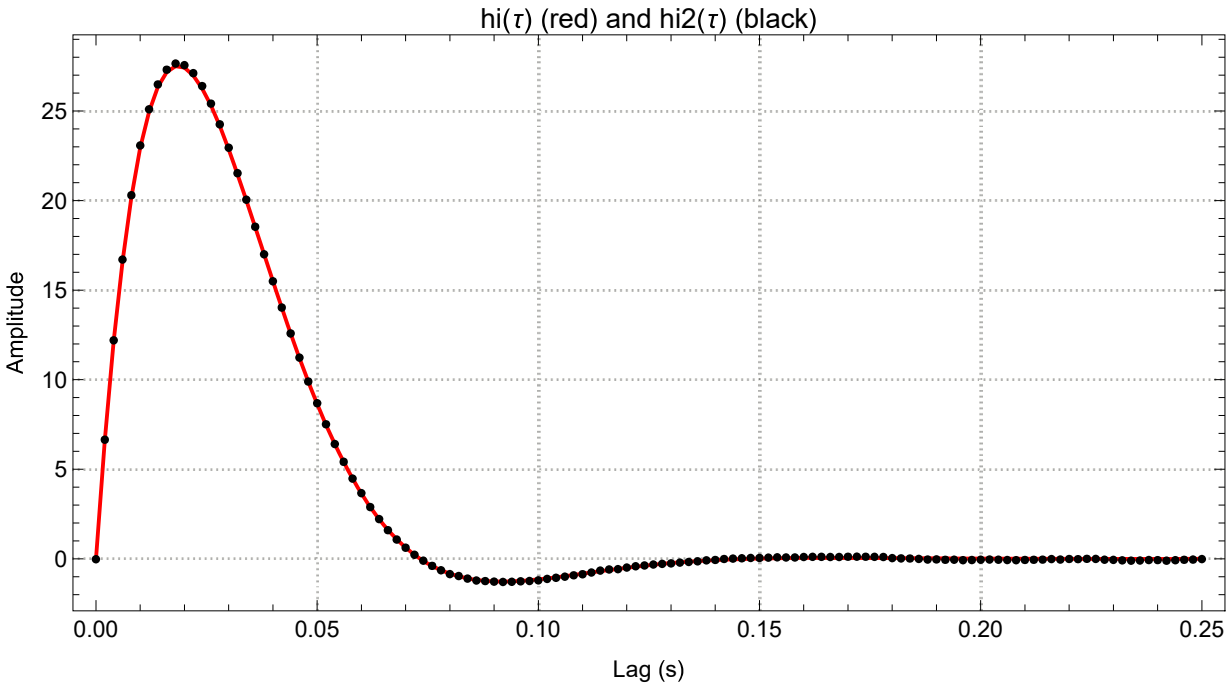
In[ ]:=
```
; u2 = n1Inv[y];
```

Re-estimate h(τ)

Now that we have u2(t) we can re-estimate h(τ):

In[ ]:=
```
■; xyccf = cor[x, u2, τMax / Δt];
hi2 = (invtoe.xyccf) / Δt;
hi2 = hi2 / (Δτ Total[hi2]); (* scales hi2 to have unity static gain *)
ListPlot[{{τi, hi}ᵀ, {τi, hi2}ᵀ}, Joined → {True, False},
 PlotTheme → "Detailed", PlotStyle → {Red, Black}, Frame → True,
 LabelStyle → {12, Black}, FrameLabel → {"Lag (s)", "Amplitude"}, AspectRatio → 0.5,
 PlotLabel → "hi(τ) (red) and hi2(τ) (black)", PlotRange → All, ImageSize → Automatic → 600]
```

Out[ ]=



Wow! Bang on.

Re-estimate u(t)

Now that we have the new impulse response function estimate, h2(t), we can convolve it with the input, x(t), to give a new estimate u3(t):

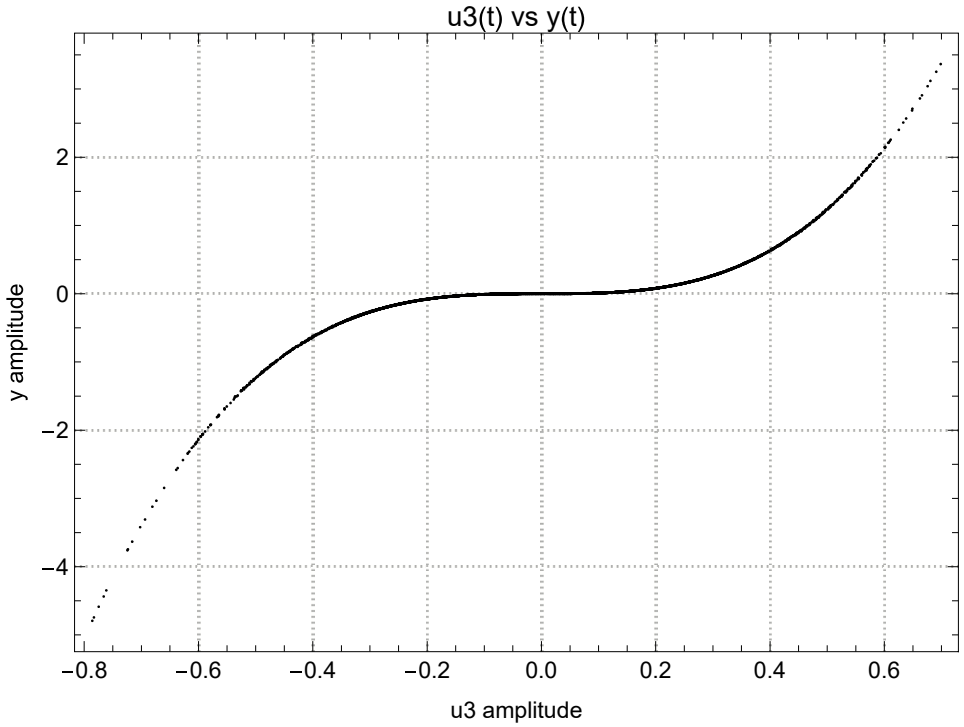
In[ ]:=
```
■; u3 = Δt ListConvolve[hi2, x, {1, 1}] ;
```

Re-estimate n(·)

Now that we have a new estimate u3(t) we can predict the form of the static nonlinearity by simply cross-plotting u3 vs y:

In[ ]:=
```
; ListPlot[{u3, y}ᵀ, PlotTheme → "Detailed", PlotStyle → {Red, Black}, Frame → True,
 LabelStyle → {12, Black}, FrameLabel → {"u3 amplitude", "y amplitude"},
 AspectRatio → 0.7, PlotLabel → "u3(t) vs y(t)", PlotRange → All, ImageSize → 500]
```

Out[ ]=



Notice that there is now much less scatter. This suggests that we are getting closer to want might be the true shape of the static nonlinearity.

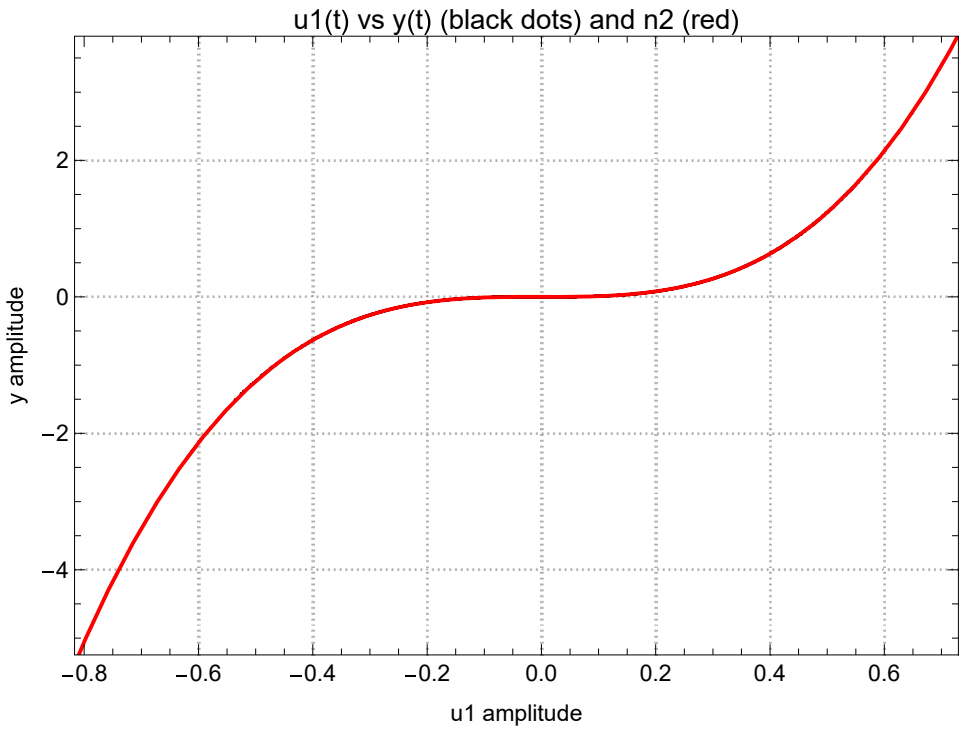
We can find an approximation to the new estimate, n2(·) by fitting a high order polynomial (again with only odd powers) to the u3(t) vs y(t) cross-plot:

In[ ]:=
```
; n2 = LinearModelFit[{u3, y}ᵀ, {1, uu, uu^3, uu^5, uu^7, uu^9}, uu]["Function"];
```

Lets plot this estimate, n2(·), of the nonlinearity together with the cross-plot of u3(t) vs y(t):

In[ ]:=
```
; Show[
 ListPlot[{u3, y}^T, PlotTheme → "Detailed", PlotStyle → Black, Frame → True,
  LabelStyle → {12, Black}, FrameLabel → {"u1 amplitude", "y amplitude"},
  AspectRatio → 0.7, PlotLabel → "u1(t) vs y(t) (black dots) and n2 (red)",
  PlotRange → All, ImageSize → 500],
 Plot[n2[uu], {uu, -1, 1}, PlotStyle → Red]]
```

Out[ ]=



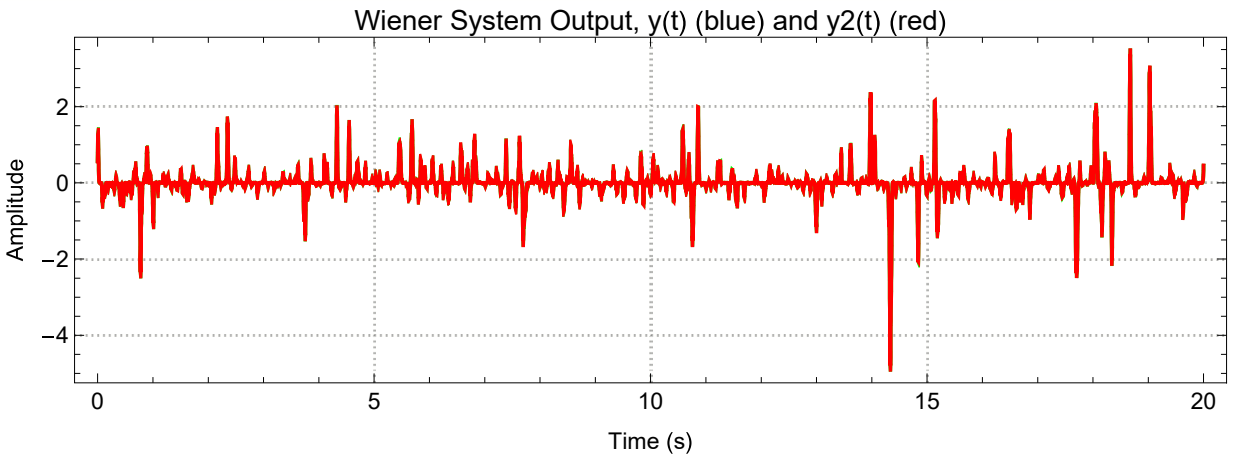
So as we can see our estimate of the Wiener system static nonlinearity, n2(·), is very close to the actual static nonlinear element, n(·).

Predict the output

Now that we have the new estimate, n2(·), we can predict the output, y2(t), from our u3(t) estimate:

In[ ]:=
```
; y2 = n2[u3];
ListLinePlot[{{ti, y}ᵀ, {ti, y2}ᵀ}, PlotTheme → "Detailed",
 PlotRange → All, PlotStyle → {Blue, Red}, Frame → True, LabelStyle → {12, Black},
 FrameLabel → {"Time (s)", "Amplitude"}, AspectRatio → 0.3, PlotLabel →
  "Wiener System Output, y(t) (blue) and y2(t) (red)", ImageSize → Automatic → 600]
```

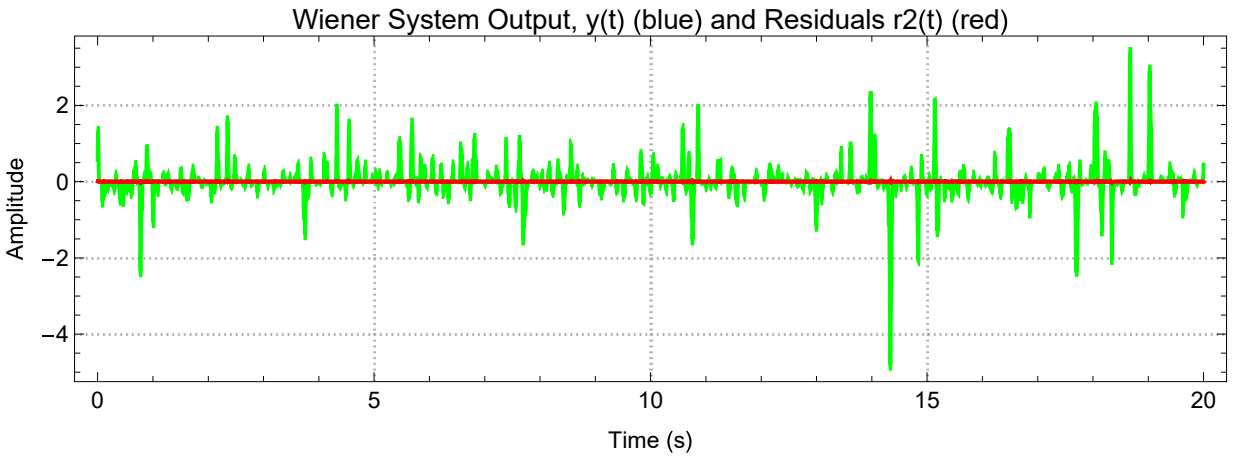
Out[ ]=



The prediction is so good it is difficult to see the difference between y(t) and y2(t). So lets calculate the residuals, r2(t):

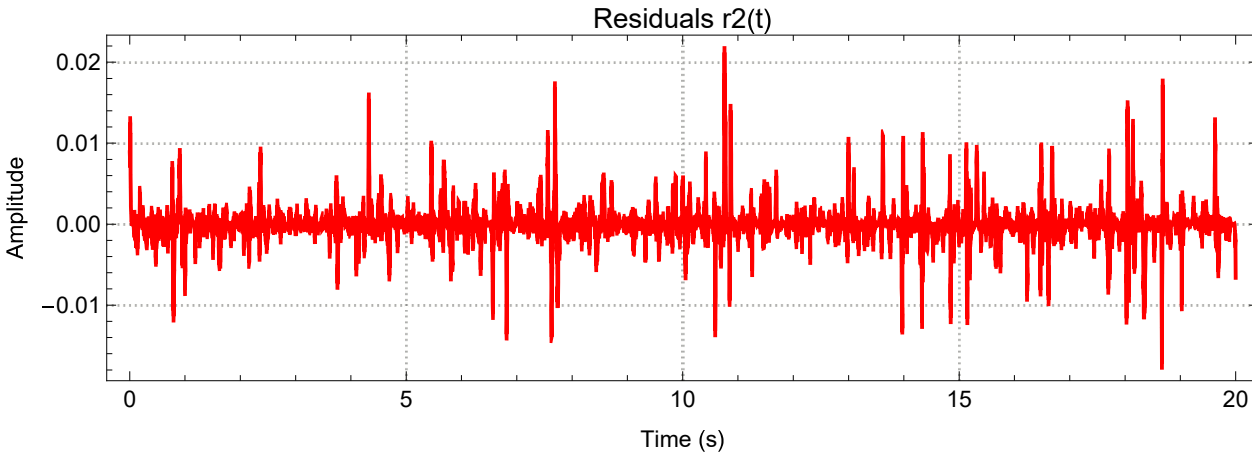
In[ ]:=
```
; r2 = y - y2;
ListLinePlot[{{ti, y}ᵀ, {ti, r2}ᵀ}, PlotTheme → "Detailed",
 PlotRange → All, PlotStyle → {Blue, Red}, Frame → True, LabelStyle → {12, Black},
 FrameLabel → {"Time (s)", "Amplitude"}, AspectRatio → 0.3,
 PlotLabel → "Wiener System Output, y(t) (blue) and Residuals r2(t) (red)",
 ImageSize → Automatic → 600]
```

Out[ ]=



In[ ]:=
```
; ListLinePlot[{ti, r2}^T, PlotTheme → "Detailed", PlotRange → All, PlotStyle → Red,
 Frame → True, LabelStyle → {12, Black}, FrameLabel → {"Time (s)", "Amplitude"},
 AspectRatio → 0.3, PlotLabel → "Residuals r2(t)", ImageSize → Automatic → 600]
```

Out[ ]=



In[ ]:=
```
; vaf2 = 100.0 (1 - Variance[r2] / Variance[y])
```

Out[ ]=

```
99.9968
```

This VAF is so high there is no point in going any further other than to generate explicit parametric models for both the impulse response function and the static nonlinearity.

As a reminder of our progress lets review the various VAFs:

In[ ]:=
```
; {vaf0, vaf1, vaf2}
```

Out[ ]=

```
{54.3523, 99.384, 99.9968}
```

Parametric models

Now that we have a good nonparametric models for the impulse response function and the static nonlinearity it is appropriate to go one more step and generate parametric (equation based) models for both.

Parametric impulse response function model

By observation of the shape of hi2(τ) it looks as though it might be well approximated (fitted) by a second-order low-pass underdamped impulse response function.

In[ ]:=
```
; h[τ_, α_, ω_, ζ_] :=
```

$$\frac{e^{-\zeta\,\tau\,\omega}\,\alpha\,\omega\,\text{Sinh}\left[\sqrt{-1+\zeta^2}\;\tau\,\omega\right]}{\sqrt{-1+\zeta^2}}$$

We start by defining an objective function to be minimized. Lets find the 3 parameters of the parametric second order low-pass underdamped impulse response function which minimize the sum of squared differences between it and the best nonparametric impulse response function model we determined, namely, h2(τ).

In[ ]:=
```
;
rmsError[α_, ω_, ζ_] :=
 With[{n = Length[hi2]}, Abs[Sqrt[((hi2 - h[τi, α, ω, ζ]).(hi2 - h[τi, α, ω, ζ])) / (n - 2)]]]
```

From inspection of h2(τ) above lets estimate that the natural frequency will be near 7 Hz. Which in rad/s is :

In[ ]:=
```
; ωω = 2 π 7.0;
```

The system is clearly only slightly underdamped (i.e., a little less than 1.0) so lets set it to be:

In[ ]:=
```
; ζζ = 0.8;
```

The static gain (i.e., overall gain) is the area under the impulse response function which we forced to be 1, so lets set it to be:

In[ ]:=
```
; αα = 1.0
```

Out[ ]=
```
1.
```

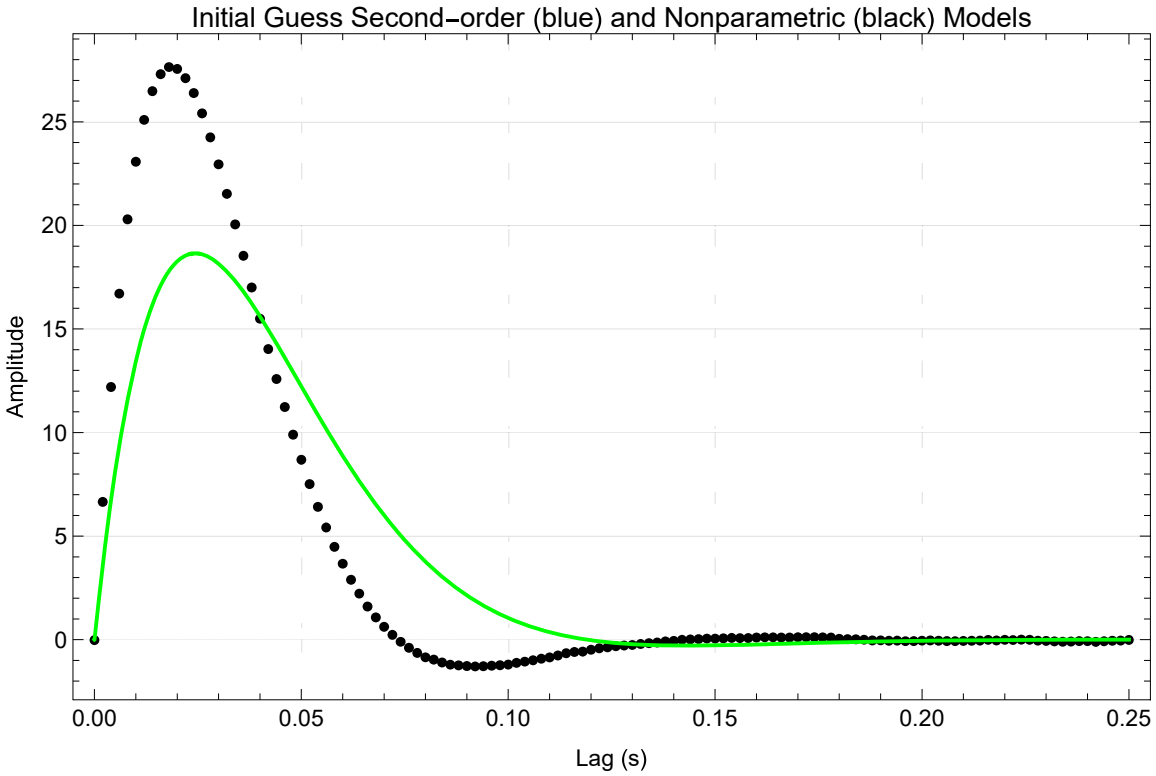
In[ ]:=
```
; rmsError[αα, ωω, ζζ] (* rms error before nonlinear least squares fit *)
```

Out[ ]=
```
3.55129
```

Now lets plot the parametric impulse response function with these estimates together with the nonparametric impulse response function, h2(τ):

In[ ]:=
```
; Show[
 ListPlot[{τi, hi2}ᵀ, PlotRange → All, Frame → True, PlotStyle → Black,
  LabelStyle → {12, Black}, GridLines → Automatic, FrameLabel → {"Lag (s)", "Amplitude"},
  PlotLabel → "Initial Guess Second-order (blue) and Nonparametric (black) Models",
  PerformanceGoal → "Quality", ImageSize → 600],
 Plot[h[τ, αα, ωω, ζζ], {τ, 0, τMax}, PlotStyle → Blue,
  PlotRange → All, PlotPoints → 1000, PerformanceGoal → "Quality"]]
```

Out[ ]=



Our initial estimates are clearly off. Lets estimate the three parameters using our trusty Levenberg-Marquardt nonlinear minimization technique:

In[ ]:=
```
;
nlm = NonlinearModelFit[{τi, hi2}ᵀ, h[τ, α, ω, ζ],
   {{α, αα}, {ω, ωω}, {ζ, ζζ}}, τ, Method → "LevenbergMarquardt"];
nlm["BestFitParameters"]
{αα, ωω, ζζ} = {α, ω, ζ} /. nlm["BestFitParameters"];
```

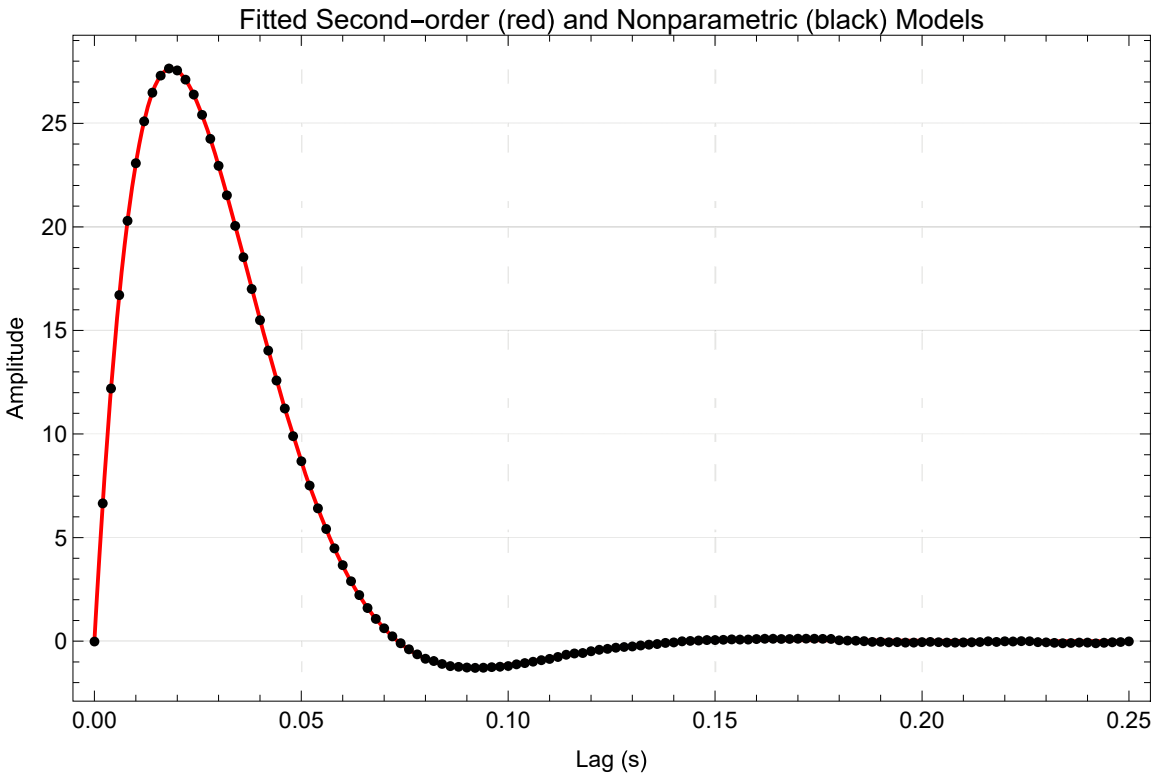
Out[ ]=
```
{α → 1.0035, ω → 60.0823, ζ → 0.700127}
```

In[ ]:=
```
; rmsError[αα, ωω, ζζ](* rms error after nonlinear least squares fit *)
```

Out[ ]=
```
0.0396973
```

In[ ]:=
```
■; Show[
 Plot[h[τ, αα, ωω, ξξ], {τ, 0, τMax}, PlotRange → All, Frame → True, PlotStyle → Red,
  LabelStyle → {12, Black}, GridLines → Automatic, FrameLabel → {"Lag (s)", "Amplitude"},
  PlotLabel → "Fitted Second-order (red) and Nonparametric (black) Models",
  PerformanceGoal → "Quality", ImageSize → 600],
 ListPlot[{τi, hi2}ᵀ, PlotStyle → Black, PlotRange → All, PerformanceGoal → "Quality"]]
```

Out[ ]=



Nice! Clearly we have an excellent fit and there is no point going to a higher order model.

The original “true” values for the static gain, natural frequency and damping were:

In[ ]:=
```
{1, 60, 0.7};
```

The estimated values are

In[ ]:=
```
■; {αα, ωω, ξξ}
```

Out[ ]=
```
{1.0035, 60.0823, 0.700127}
```

These are almost exactly the same. So we have identified the parametric dynamic linear component of the Wiener system.

Parametric static nonlinear element model

It remains for us to find a parametric model for the static nonlinearity. The last estimate of the static nonlinearity was n2. Mathematica provides a function called FindFormula[ ] which attempts to discover static models which fit a data set. It takes a while because it is trying a huge number of candidate

models:

```
In[ ]:= ■; fit = FindFormula[{u3, y}ᵀ, uu, 10, All ]
```

Out[ ]=

	Score	Error	Complexity
$9.88505\ uu^{3.}$	12.4181	0.00000398019	7
$-0.000485649\ uu + 0.0014246\ uu^2 + 9.88712\ uu^3$	12.3904	0.00000397118	20
$1.29937\ uu$	2.95791	0.0514511	4
$1.36923\ uu$	2.95555	0.051573	4
$0.00592146 + 1.31355\ uu$	2.95185	0.0514076	7
$-1.95575 + 1.94357 \times 1.94131^{uu}$	2.94666	0.0513194	10
$1.17481\ uu$	2.9425	0.0522505	4
0	2.08619	0.123874	1
1.90786^{uu}	−0.0819769	1.07548	4
$2.^{uu}$	−0.0820062	1.07551	4

This discovery function returns suitable static models together with a “Score” and a measure of “Complexity”. A “good” model is one which has a high Score (a good fit) but a low level of complexity under the assumption that simplicity is important. Using these two criteria we choose the simple cuber model. A cuber is exactly the static nonlinearity used in the Wiener model we used. Furthermore the single parameter of this model has a value near to 10 which is the same as that used when the Wiener system was specified previously. In other words we have identified the static nonlinearity.

FindFormula does a quick fit and test of a wide range of functions. Lets redo the pure cuber fit:

```
In[ ]:= ■; n3 = LinearModelFit[{u3, y}ᵀ, {1, uu^3}, uu]["BestFitParameters"];
        n3[[2]]
```

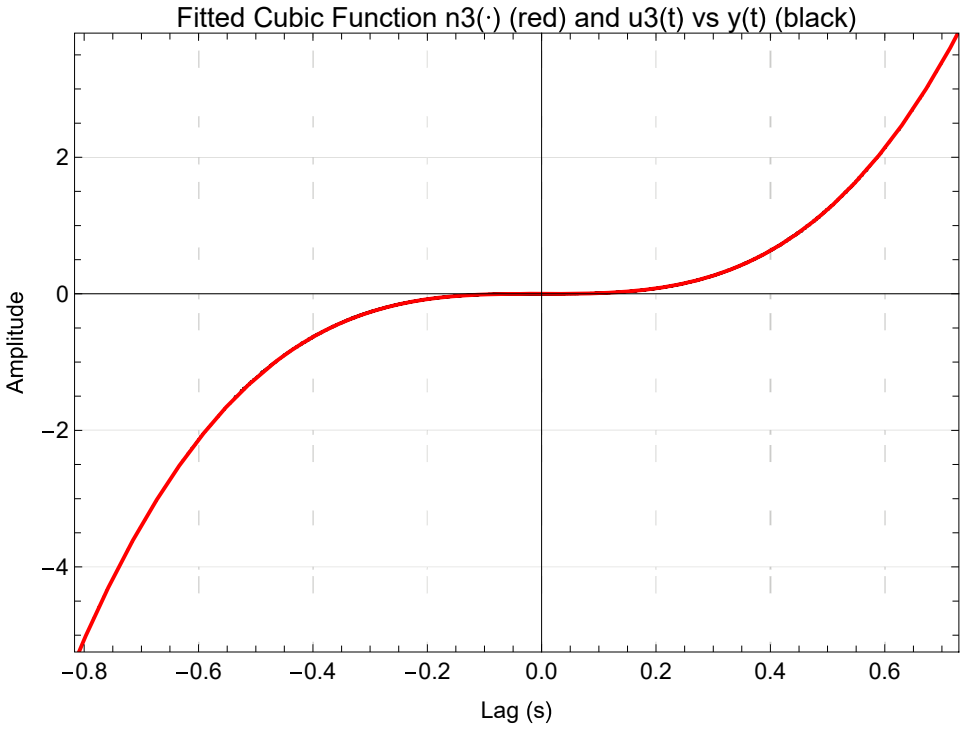
Out[ ]=

```
9.88505
```

The scaling parameter is similar to the one returned by FindFormula[ ] and is close to the “true” value of 10 used in the Wiener system simulation.

In[ ]:=
```
■; Show[
 ListPlot[{u3, y}^T, PlotStyle → Black, PlotRange → All, Frame → True, PlotStyle → Red,
  LabelStyle → {12, Black}, GridLines → Automatic, FrameLabel → {"Lag (s)", "Amplitude"},
  PlotLabel → "Fitted Cubic Function n3(·) (red) and u3(t) vs y(t) (black)",
  AspectRatio → 0.7, PerformanceGoal → "Quality",
  ImageSize → 500, PerformanceGoal → "Quality"],
 Plot[n3[[2]] uu^3, {uu, -1, 1}, PlotRange → All, PlotStyle → Red]]
```

Out[ ]=



Conclusion

In conclusion we have managed to identify both the linear dynamic part and the static nonlinear part of the the Wiener system.

The Wiener system has been identified both non-parametrically as well as parametrically!!